hhu

hhu
Heinrich Heine
Universität Düsseldorf

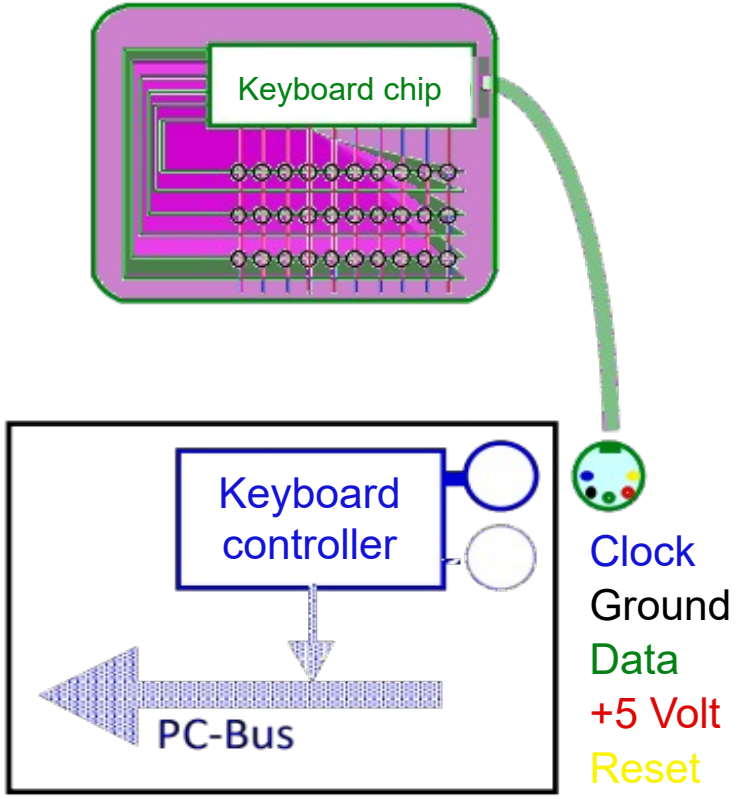
HeineOS

Assignment material: Keyboard

Prof. Dr. Michael Schöttner
Dr. Fabian Ruhland

PS/2 Keyboard - Components

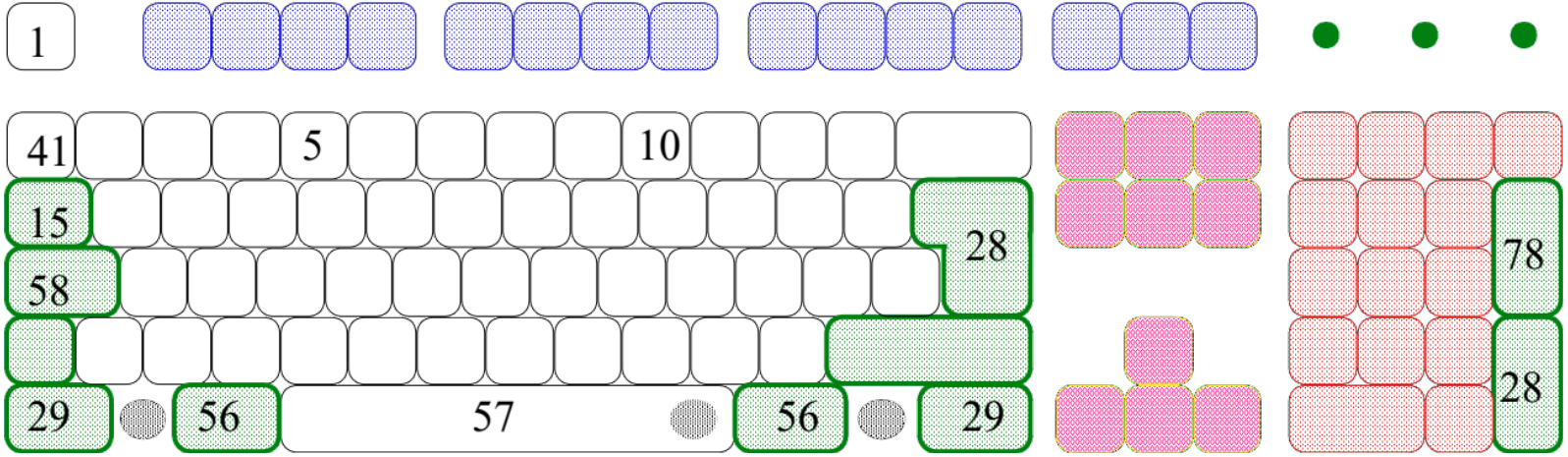
- Keyboard chip inside the keyboard:
  - Serial communication with the keyboard controller
  - Queries the keyboard via its **scan matrix**
- Keyboard controller on the mainboard:
  - Nowadays, integrated inside the *South Bridge*
  - Additional port for a PS/2 mouse
  - Uses **IRQ1** and IO-Ports **0x60** and **0x64**
- Modern PC: USB keyboard:
  - USB Legacy Support: Backwards compatible communication via the PS/2 controller



PS/2 Keyboard - Scancodes

Heinrich Heine Universität Düsseldorf

- Keyboards send s**cancodes** (7-bit unique number for each key)
- Driver needs to translate scancodes into characters (according to the configured layout)
- Lock keys functionality (e.g. Caps Lock) is also implemented inside the driver
- Example for an *MF II* keyboard:



PS/2 Keyboard - Scancodes

- Example: Scancodes and ASCII-codes:

Char	ASCII
(	40
0	48
1	49
2	50
A	65
B	66
a	97

Key	Scancode
A	30
a	30
S	31
D	32
Cursor Up	72
Cursor Down	80
a	97

- Keyboard controller sends additional information
  - **Make-Code** when pressing/holding a key
  - **Break-Code** when releasing a key

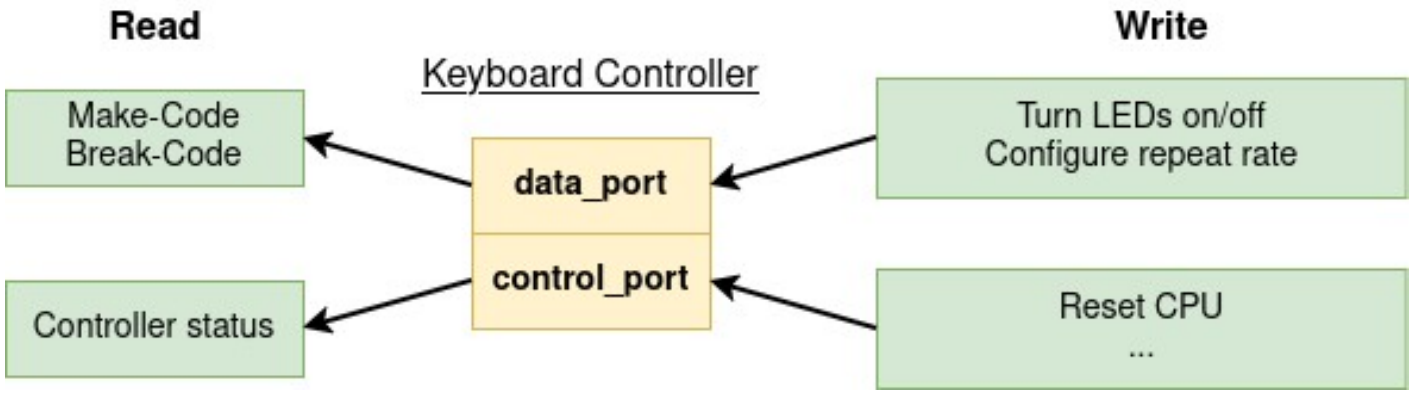
PS/2 Keyboard - Scancodes

Heinrich Heine Universität Düsseldorf

- Usually, the following applies:
  - **Make-Code** (key pressed) → Scancode
  - **Break-Code** (key release) → Scancode + 128 (Bit 7 is set)
- However, some keys send multiple codes
  - For example the function keys (F1-F12)
  - Up to three codes per key
- Builtin repeat functionality
  - Hardware sends Make-Codes multiple times, when a key is held down
- Decoding is cumbersome
  - HeineOS already contains implemented function `fn decode_byte(&mut self, mut code: u8) -> bool`
  - Returns true if a key has been fully decoded, false if more codes are expected by the controller
  - Decoded key is stored in variable `Keyboard::gather`

PS/2 Keyboard - Communication

- Communication with the keyboard controller works via two I/O-ports:
  - Input/Output register (data_port) **0x60**
  - Status/Control register (control_port) **0x64**



PS/2 Keyboard – Status Register

Heinrich Heine Universität Düsseldorf

- Access by **reading** from I/O-port **0x64**
  - Bit 7: PARE Parity error
  - Bit 6: TIM Response timeout
  - Bit 5: AUXB Byte available vom auxiliary device (mouse) → Please read byte from output buffer
  - Bit 4: KEYL Keyboard available (1) or locked (0)
  - Bit 3: C/D Command or data byte written
  - Bit 2: SYSF Self test successful
  - Bit 1: INPB Input buffer busy → Please wait
  - Bit 0: OUTB Byte available → Please read byte from output buffer

PS/2 Keyboard – Control Register

- Access by **writing** to I/O-port **0x64**
  - Keyboard controller commands (1 byte)
    - 0xAA Self test
    - 0xAE Enable keyboard
    - 0xED Turn on/off LEDs
    - 0xF3 Set repeat rate
  - **Caution:** Before sending a command, make sure the input buffer is empty
    → Check **Bit 1 (INPB)** in status register
- An overview of keyboard commands is given here:
  https://wiki.osdev.org/PS/2_Keyboard

PS/2 Keyboard – Reading Keys

Heinrich Heine Universität Düsseldorf

- Call `Keyboard::poll_key_event()`
  - Calls `Keyboard::try_read_next_byte()` in a loop until a key is returned
    - `try_read_next_byte()` checks if a byte is available in the output buffer by checking **BIT 0 (OUTB)** in the status register
    - If a byte is available it is decoded using `Keyboard::decode_byte()`
    - If `decode_byte()` returns `true` return `Keyboard::gather`, otherwise return `None`
- Actively reading the keyboard in a loop (**polling**) is rather inefficient
  - Will be replaced by an interrupt based implementation later on in the project

PS/2 Keyboard – Send Commands

- General sequence:
  - Wait until the input buffer is empty → Check **Bit 1 INPB** in status register
  - Write the command byte to the data port
  - Wait for an answer → Check Bit **0 OUTB** in status register
  - Read answer from data port
  - If the answer is not an ACK (0xFA) abort and return -1
  - Otherwise write parameter byte to the data port
  - Wait for an answer → Check Bit **0 OUTB** in status register
  - If the answer is an ACK (0xFA) return 0, othwerwise return -1

PS/2 Keyboard – Control LEDs

- Structure of the parameter byte:

Bit	Meaning
7	-
6	-
5	-
4	-
3	-
2	Caps Lock
1	Num Lock
0	Scroll Lock

PS/2 Keyboard – Repeat Rate

- **Typematic rate:**
  - Determines how fast the same scan code is sent, when a key is held down
  - Simulates multiple key presses
  - Repeat rate is configurable between 2 and 30 repetitions per second
- **Delay rate:**
  - Determines the timespan, after which the repetition should start (tolerance ca. 20%)

Code	Delay
00	¼ s
01	½ s
10	¾ s
11	1 s

PS/2 Keyboard – Repeat Rate

Heinrich Heine Universität Düsseldorf

- **Typematic rate:**

Code	Repetitions	Code	Repetitions	Code	Repetitions	Code	Repetitions
11111	2.0s	10111	4.0s	01111	8.0s	00111	16.0s
11110	2.1s	10110	4.3s	01110	8.6s	00110	17.1s
11101	2.3s	10101	4.6s	01101	9.2s	00101	18.5s
11100	2.5s	10100	5.0s	01100	10.0s	00100	20.0s
11011	2.7s	10011	5.5s	01011	10.9s	00011	21.8s
11010	3.0s	10010	6.0s	01010	12.0s	00010	24.0s
11001	3.3s	10001	6.7s	01001	13.3s	00001	26.7s
11000	3.7s	10000	7.5s	01000	15.0s	00000	30.0s

- The codes are calculated using the following formula:
  Repetitions per second = $1 / ((8 + A) * 2^B * 0.00417)$
  A = Bits 0-2 of code; B = Bits 3-4 of code

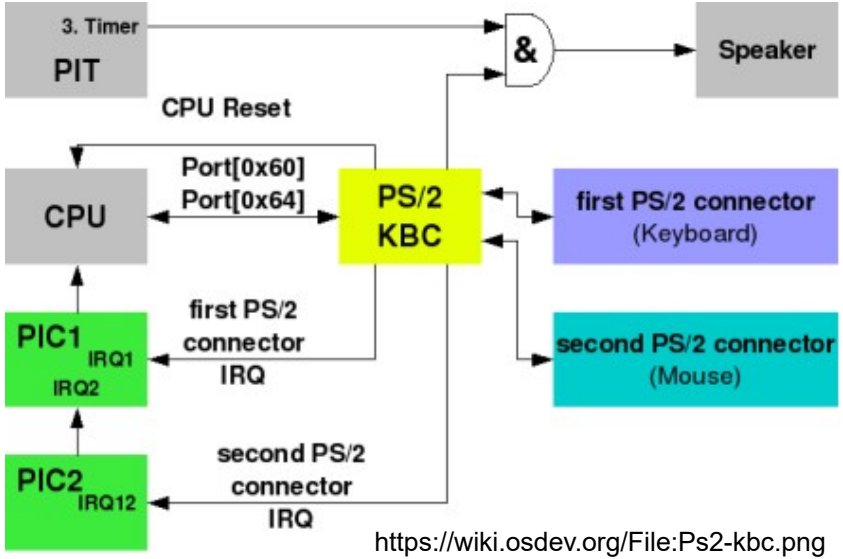
PS/2 Keyboard – Repeat Rate

- Structure of the full parameter byte:

Bit	Meaning
7	-
6	Delay rate
5	Delay rate
4	Typematic rate
3	Typematic rate
2	Typematic rate
1	Typematic rate
0	Typematic rate

PS/2 Keyboard – Mouse support

- Both the mous and the keyboard are connected to the PS/2 controller
- Keyboard uses **IRQ1** and mous **IRQ12**
- Whether a data byte came from the keyboard or the mouse is signaled via the status register (**Bit 5 AUXB**)



https://wiki.osdev.org/File:Ps2-kbc.png